Mutexes And Semaphores

Jibon Naher[1] and Samnun Azfar[2]

[1]Lecturer, CSE
[2]Junior Lecturer,CSE

February 26, 2026

1 Pointer Safety and Multiplexing

Part A (Mutex): Hot-Swappable Configuration System

Problem Statement
A server application uses a global configuration object.

- **Reader Threads:** Continuously access values from the current configuration.
- **Updater Thread:** Periodically generates a completely new configuration object and replaces the old one.

Rules

- Readers must never access invalid or freed memory.
- The switch from old config to new config must be atomic.
- Old configuration memory must be safely freed only after the swap is secured.

Constraints

- Use **one mutex** only.
- Do not use Read-Write locks.

Hints

- Use an int pointer variable as the config object.
- Write one update function which calls `free()` on the pointer variable and then uses `malloc()` to allocate new memory for the pointer. Then set the pointer to some value.
- Write multiple reader functions and run them in separate threads, then print the value of the pointer.
- Use `stdlib.h` for memory allocation functions and `unistd.h` to use the `sleep()` functions to make the printings look sane in terminal.

Invariant to Preserve

$$\text{active\_config} \neq \text{NULL} \quad (\text{At all times})$$

Part B (Semaphore): Dining Philosophers (Limited Seating)

Problem Statement
5 Philosophers sit around a table with 5 forks. To eat, a philosopher needs both the left and right forks.

Deadlock Prevention Strategy
Instead of complex fork logic, you must restrict the number of philosophers allowed at the table simultaneously.

- Max **4 philosophers** allow to sit at the table at once.
- If 4 are seated, at least one is guaranteed to be able to pick up both forks.

Constraints

- Use **one counting semaphore** to represent the room capacity.
- Use mutexes/semaphores for individual forks as needed.

2 Complex Buffering and Barriers

Part A (Mutex): Thread-Safe Histogram (Sharded Updates)

Problem Statement
A large dataset is processed by multiple threads. They update a shared frequency array (histogram) of size 10.

```
histogram[10] = {0, 0, ...}
```

Rules

- Input is an array of n integers which have values k ranging from $0 < k < 10$. k is an integer.
- Assume you have $n = 9$. If you create 2 threads then thread $t1$ will calculate the histogram for indices $0 - 4$ and thread $t2$ will calculate the histogram for indices $5 - 8$, both done in a historgam array local to their threads. Then both the threads concurrently update the global histogram using the locally generated histograms.

Constraints

- Use **one mutex**.

Invariant to Preserve

$$\sum \text{histogram}[i] = n$$

Part B (Semaphore): H_2O Bond Formation (Barrier)

Problem Statement
A water molecule is formed by 2 Hydrogen atoms and 1 Oxygen atom. Threads represent individual atoms.

Rules

- H atoms must wait for another H and one O.

- O atoms must wait for two H atoms.
- They must ”bond” (exit the barrier) simultaneously in groups of three (H-H-O).
- Hint: You can concurrently create H but not O.

Constraints

- Use **semaphores only**.
- No mutexes.
- Ensure no atom is left behind or grouped incorrectly.

Expected Execution

```
H Ready -> H Ready -> O Ready -> Bond Formed!
```